

# DeerBVP: гибридный ИИ-решатель краевых задач

Черных Александр, МАИ, 3 курс

Статус: исследовательский проект

## Аннотация

В работе представлен гибридный подход к решению нелинейных краевых задач второго порядка с малым параметром при старшей производной. Предлагается нейросетевая модель BVPNetTurbo, которая предсказывает 16 коэффициентов кубических В-сплайнов по 25 параметрам задачи. Это начальное приближение используется в классическом итерационном решателе `scipy.solve_bvp`. Показано, что предложенный метод существенно расширяет область сходимости численного алгоритма, особенно в режиме сингулярного возмущения, когда стандартное линейное приближение приводит к расходимости. Метод демонстрирует устойчивость даже при значениях параметра  $\varepsilon$  на порядок меньше обучающих.

## 1 Постановка задачи

### 1.1 Математическая формулировка

Рассматривается нелинейная краевая задача (BVP) второго порядка на отрезке  $x \in [0, 1]$ :

$$\varepsilon y'' + p(x) y' + q(x) y + j y^2 + k y^3 = f(x) \quad (1)$$

$$y(0) = \alpha, \quad y(1) = \beta \quad (2)$$

Коэффициенты задаются параметрически:

$$p(x) = p_0 + p_1 x + p_2 x^2 + w_1 \sin(w_2 x) + v_1 \cos(v_2 x) \quad (3)$$

$$q(x) = q_0 + q_1 x + q_2 x^2 + e_1 \sin(e_2 x) + u_1 \cos(u_2 x) \quad (4)$$

$$f(x) = A \exp\left(-\left(\frac{x - \mu}{\sigma}\right)^2\right) + c_0 + c_1 x + c_2 x^2 \quad (5)$$

Итого **25 входных параметров**, определяющих конкретный экземпляр задачи.

## 1.2 Физический смысл и трудность

Параметр  $\varepsilon \in [10^{-4}, 10^{-1}]$  — малый параметр при старшей производной. При  $\varepsilon \rightarrow 0$  возникает **сингулярное возмущение**: решение образует **погранслои** — область резкого изменения функции шириной порядка  $\varepsilon$  у одной из границ.

Классический итерационный решатель `scipy.solve_bvp` требует начального приближения. Стандартный подход — линейная интерполяция между граничными значениями:

$$y_{\text{zero}}(x) = \alpha + (\beta - \alpha)x \quad (6)$$

При малых  $\varepsilon$  такое приближение не содержит структуры погранслоя, и итерационный процесс **расходится** — решатель возвращает ошибку.

## 1.3 Цель проекта

Обучить нейросеть, которая по 25 параметрам задачи предсказывает **структурно правильное начальное приближение** — такое, которое уже содержит погранслои и выводит решатель в область сходимости.

## 2 Описание метода

### 2.1 Общая идея: гибридный подход

В основе метода лежит компромисс между классическими численными алгоритмами и глубинным обучением. Вместо того чтобы обучать нейросеть решать дифференциальное уравнение напрямую (как в PINNs), мы используем нейросеть исключительно для генерации *информированного начального приближения*. Это приближение затем уточняется стандартным итерационным решателем `scipy.solve_bvp`, который гарантирует высокую точность и сходимость при достаточном качестве старта.

Такой подход имеет ряд преимуществ:

- Сохраняются все гарантии численного метода (оценка погрешности, устойчивость);
- Нейросеть не обязана выдавать решение с высокой точностью — достаточно захватить структуру (погранслои);
- Обучение происходит на метках, полученных тем же численным методом, что исключает накопление ошибок;
- Модель может обобщаться на более жёсткие режимы, чем обучающие, за счёт изучения структурных закономерностей.

Формально алгоритм работы с новой задачей выглядит так:

1. Поступает вектор параметров  $\theta = (\varepsilon, p_0, \dots, c_2) \in \mathbb{R}^{25}$ ;
2. Нейросеть  $F(\theta)$  отображает его в 16 коэффициентов  $\mathbf{c} = (c_1, \dots, c_{16})$ ;

3. Строится начальная функция  $y_0(x) = \sum_{i=1}^{16} c_i B_i(x)$ , где  $B_i(x)$  — кубические В-сплайны на равномерной сетке;
4. Решатель `solve_bvp` получает  $y_0(x)$  и  $y'_0(x)$  (аналитически) и выполняет итерации до достижения заданной точности.

## 2.2 Представление решения через В-сплайны

Выбор В-сплайнов обусловлен несколькими соображениями:

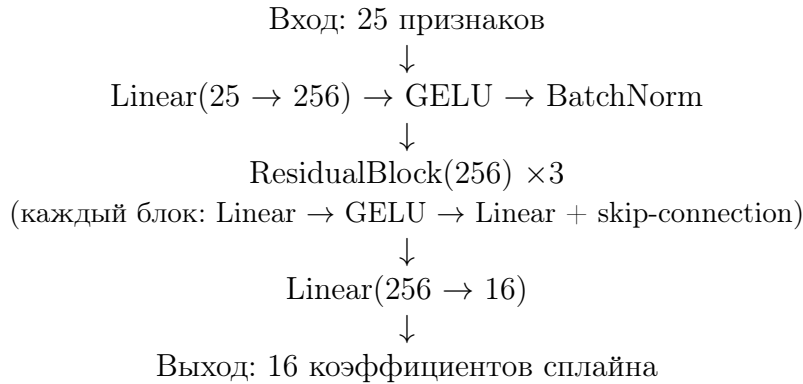
- **Компактность:** 16 коэффициентов кодируют гладкую функцию на  $[0, 1]$  с достаточной точностью для класса задач (типичные решения содержат один резкий переход);
- **Гладкость:** В-сплайны имеют непрерывные производные до второго порядка, что необходимо для вычисления невязки и передачи  $y'$  в солвер;
- **Линейность по коэффициентам:** нейросеть предсказывает линейные веса, что упрощает обучение регрессии.

Используются кубические В-сплайны с равномерными узлами, количество которых (16) подобрано эмпирически: меньшее число не воспроизводит погранслои, большее — избыточно и увеличивает размерность задачи.

Таким образом, вместо предсказания 500 значений на сетке мы сводим задачу к регрессии 16 чисел, что значительно уменьшает сложность обучения и позволяет нейросети эффективно усваивать закономерности.

## 2.3 Архитектура нейросети (BVPNetTurbo)

Модель BVPNetTurbo представляет собой полносвязную нейронную сеть с пропускными соединениями (residual connections). На вход подаётся вектор из 25 нормализованных признаков, соответствующих параметрам краевой задачи. Первый слой расширяет размерность с 25 до 256 нейронов, после чего применяется функция активации GELU (Gaussian Error Linear Unit) и пакетная нормализация (BatchNorm). Далее следуют три идентичных остаточных блока, каждый из которых состоит из двух полносвязных слоёв по 256 нейронов с активацией GELU между ними; выход блока суммируется с его входом (skip connection), и результат снова пропускается через GELU. Завершающий слой линейно отображает 256 признаков в 16 выходных коэффициентов кубических В-сплайнов; выходной слой не содержит активации, чтобы не ограничивать диапазон предсказываемых значений. Общее число обучаемых параметров составляет 406 032.



**Ключевые решения и их обоснование.** Ниже перечислены архитектурные выборы вместе с честной оценкой того, насколько каждый из них подтверждён экспериментально в рамках проекта, а насколько опирается на общие соображения из литературы.

[leftmargin=\*]

- **GELU вместо ReLU.** Гладкая активация в целом лучше подходит для регрессии непрерывных физических величин — это общее наблюдение, многократно подтверждённое в литературе по регрессионным сетям. Прямого А/В-сравнения GELU против ReLU на данном датасете не проводилось.
- **Residual (skip) connections.** В классических работах (ResNet и последующие) показано, что такие соединения предотвращают затухание градиентов и облегчают обучение глубоких сетей. В рамках проекта архитектура с residual-блоками тестировалась против варианта без них лишь косвенно — через серию экспериментов с изменением размера и глубины сети (см. раздел 2.4.1), где skip-connections присутствовали во всех вариантах. Изолированного сравнения «с skip-connections / без них» при прочих равных не проводилось, поэтому прирост качества именно от этого компонента архитектуры эмпирически не выделен.
- **BatchNorm только во входном слое.** Такая конфигурация дала наилучший результат по сравнению с альтернативами, опробованными в ходе подбора (см. раздел 2.4.1). Гипотеза о том, что нормализация внутри остаточных блоков «маскирует взаимосвязи», правдоподобна, но отдельно не проверялась — решение принято по результату, а не по диагностике причины.
- **Без Dropout.** В нескольких прогонах с разным dropout rate (0.1 и выше) итоговый  $R^2$  либо не менялся, либо немного снижался; лучший результат стабильно давала конфигурация без Dropout. Это эмпирическое наблюдение для конкретного датасета, а не общее утверждение об отсутствии шума в обучающих данных.

Таким образом, итоговая архитектура — результат эмпирического подбора (см. раздел 2.4.1) в сочетании с общими рекомендациями из литературы по регрессионным нейросетям, а не строго доказанных по отдельности проектных решений.

## 2.4 Обучение и настройка

### 2.4.1 Настройка гиперпараметров

Выбор окончательной архитектуры и гиперпараметров осуществлялся на основе серии экспериментов на валидационной выборке (10% от обучающей). Перебирались следующие параметры:

- глубина сети: от 1 до 5 остаточных блоков;
- ширина скрытых слоёв: 128, 256, 512;
- функция активации: ReLU, GELU, Swish;
- наличие/отсутствие BatchNorm на разных слоях;
- скорость обучения:  $10^{-4}$ ,  $3 \cdot 10^{-4}$ ,  $10^{-3}$ ,  $3 \cdot 10^{-3}$ ;
- использование Dropout с вероятностями 0.0, 0.1, 0.2.

Каждая конфигурация обучалась на 200 эпох с фиксированным seed. Критерием выбора служило значение  $R^2$  на валидационной выборке, а также стабильность сходимости. Оптимальной оказалась конфигурация с тремя блоками по 256 нейронов, GELU, BatchNorm только на входе, без Dropout и с learning rate  $10^{-3}$ .

### 2.4.2 Процесс обучения

Обучение проводится в два этапа:

1. **Нормализация данных:** входные признаки стандартизируются (StandardScaler), а целевые коэффициенты — с использованием RobustScaler, устойчивого к возможным выбросам в метках;
2. **Минимизация среднеквадратичной ошибки (MSE)** между предсказанными и истинными коэффициентами сплайнов.

Используется оптимизатор AdamW с начальной скоростью обучения  $10^{-3}$ , планировщик CosineAnnealingLR на 500 эпох и батч-размер 128. Обучение останавливается досрочно при отсутствии улучшений на валидационной выборке в течение 100 эпох.

Выбор MSE обусловлен тем, что ошибка в коэффициентах напрямую влияет на ошибку функции (в силу линейности базиса), и минимизация MSE коэффициентов эквивалентна минимизации среднеквадратичного отклонения функции при равномерной сетке.

## 2.5 Почему это работает: интуиция

Нейросеть учится отображать 25 физических параметров в коэффициенты сплайнов, которые аппроксимируют решение. Это отображение не является тривиальным, но глубокая сеть с пропускными связями способна его выучить благодаря наличию достаточно репрезентативного датасета (более 35 000 примеров). Важно, что решение для разных

параметров меняется плавно — это свойство позволяет сети обобщать на новые комбинации.

При этом сеть не обязана выдавать идеальное решение — достаточно, чтобы оно содержало грубую структуру (например, правильное положение и крутизну погранслоя). Именно это свойство делает начальное приближение «хорошим» для итерационного метода: солвер стартует из области притяжения решения, а не из области, где невязка велика и итерации расходятся.

## 3 Датасет

### 3.1 Генерация

Параметры задач сэмплировались методом случайного равномерного сэмплирования из физически обоснованных диапазонов. Запущено 50 000 попыток.

Каждая задача решалась **двухэтапно** для обеспечения высокой точности меток:

1. Этап 1:  $\text{tol}=1\text{e-}3$  от линейного старта  $\rightarrow$  грубое решение;
2. Этап 2:  $\text{tol}=1\text{e-}5$  от грубого решения  $\rightarrow$  точное решение.

Такой подход позволяет получать метки высокой точности даже для задач, где прямой старт с  $\text{tol}=1\text{e-}5$  не сходится.

### 3.2 Статистика

Статистика датасета приведена в таблице 1.

Таблица 1: Статистика датасета

| Показатель                               | Значение                   |
|------------------------------------------|----------------------------|
| Попыток генерации                        | 50 000                     |
| Успешно решено                           | 36 141 (72.3%)             |
| После фильтрации выбросов ( $ c  > 10$ ) | <b>35 952</b>              |
| Удалено как выбросы                      | 189 (0.5%)                 |
| Обучающая выборка (85%)                  | 30 559                     |
| Тестовая выборка (15%)                   | 5 393                      |
| Сетка решения                            | 500 равномерных узлов      |
| Точность меток                           | $\text{tol} = 1\text{e-}5$ |

### 3.3 Диапазоны параметров

Диапазоны варьируемых параметров задачи представлены в таблице 2.

Таблица 2: Диапазоны параметров задачи

| Параметр        | Диапазон                     | Примечание                |
|-----------------|------------------------------|---------------------------|
| $\varepsilon$   | $[10^{-4}, 5 \cdot 10^{-2}]$ | в лог-шкале при обучении  |
| $\alpha, \beta$ | $[-2, 2]$                    | граничные условия         |
| $p_0, p_1, p_2$ | $[-3, 3]$                    | коэффициенты переноса     |
| $q_0, q_1, q_2$ | $[-3, 3]$                    | коэффициенты реакции      |
| $j$             | $[-0.8, 0.8]$                | квадратичная нелинейность |
| $k$             | $[-0.8, 0.8]$                | кубическая нелинейность   |
| $A$             | $[-4, 4]$                    | амплитуда источника       |

## 4 Результаты

### 4.1 Качество модели

Метрики на тестовой выборке в **реальном пространстве** (не в scaled) приведены в таблице 3.

Таблица 3: Метрики качества модели на тестовой выборке

| Метрика                      | Значение              |
|------------------------------|-----------------------|
| MSE                          | $2.22 \times 10^{-1}$ |
| RMSE                         | $4.71 \times 10^{-1}$ |
| $R^2$ (суммарный)            | 0.863                 |
| $R^2$ (крайние коэффициенты) | 0.92–0.94             |
| $R^2$ (средние коэффициенты) | 0.84–0.87             |

$R^2$  крайних коэффициентов выше — они кодируют граничные условия, которые задаются явно. Средние коэффициенты отвечают за форму решения внутри отрезка — более сложная для предсказания задача.

### 4.2 Ключевой результат: расширение области сходимости

Проведено два независимых теста по **5000 случайных задач** с  $\text{tol}=1\text{e-}5$ ,  $\text{seed}=42$ .

#### 4.2.1 Тест 1 — штатное распределение (in-distribution)

Параметры из того же распределения, что и обучающая выборка. Результаты представлены в таблице 4.

#### 4.2.2 Тест 2 — стресс-тест (out-of-distribution)

$\varepsilon$  форсировано в диапазон  $[5 \cdot 10^{-5}, 8 \cdot 10^{-4}]$  — жёстче обучающего на порядок. Результаты — в таблице 5.

Таблица 4: Результаты теста 1 (in-distribution)

| Показатель                                  | Zero (линейный) | DeepBVP         |
|---------------------------------------------|-----------------|-----------------|
| Success Rate (все задачи)                   | 63.8%           | <b>74.9%</b>    |
| Success Rate (решаемые задачи) <sup>1</sup> | 84.6%           | <b>99.3%</b>    |
| Спасённых задач                             | —               | <b>581</b>      |
| Потерянных задач                            | —               | 25              |
| Соотношение побед/поражений                 | —               | <b>23.2 : 1</b> |
| Среднее итераций                            | 3.5             | 3.5             |

Таблица 5: Результаты теста 2 (out-of-distribution)

| Показатель                                  | Zero (линейный) | DeepBVP         |
|---------------------------------------------|-----------------|-----------------|
| Success Rate (все задачи)                   | 29.9%           | <b>39.4%</b>    |
| Success Rate (решаемые задачи) <sup>1</sup> | 74.1%           | <b>97.8%</b>    |
| Спасённых задач                             | —               | <b>521</b>      |
| Потерянных задач                            | —               | 44              |
| Соотношение побед/поражений                 | —               | <b>11.8 : 1</b> |
| Среднее итераций                            | 5.9             | 5.9             |

<sup>1</sup> *Conditional SR* — считается только по задачам, где `scipy.solve_bvp` нашёл решение хотя бы с одним из двух начальных приближений при фиксированных настройках ( $tol=1e-5$ , сетка 500 узлов). Задачи из категории "оба упали" не исключаются как математически неразрешимые — возможно, они допускают решение при других параметрах солвера или другом начальном приближении. Это ограничение протокола тестирования, а не утверждение о существовании решения. Формула: (оба сошлись + только NN) / (все — оба упали).

### 4.3 Интерпретация результатов

**Главный вывод:** DeepBVP не ускоряет решение там, где и так сходится — среднее число итераций одинаково. Ценность метода — в **расширении области сходимости**: задачи, для которых классический старт приводит к расходимости, решаются с нейросетевым приближением.

**Масштаб эффекта:** на 5000 задач нейросеть спасла 581 случай в штатном режиме и 521 в стресс-тесте при единичных поражениях (25 и 44 соответственно).

**Обобщение за пределы обучения:** в стресс-тесте модель работает при  $\varepsilon \approx 6 \cdot 10^{-5}$  — значении, которого не было в обучающей выборке (минимум  $10^{-4}$ ). Conditional SR остаётся на уровне 97.8% — сеть выучила структурные закономерности (наличие погранслоя, его положение и крутизну), а не просто запомнила примеры.

**Надёжность:** соотношение побед к поражениям 23.2:1 в штатном режиме означает, что нейросетевой старт статистически безопасен — случаи, когда он хуже линейного, единичны.



## 5 Сравнение с альтернативными подходами

### 5.1 Почему не классический ML

В ходе предварительных экспериментов проверялись Random Forest, XGBoost, CatBoost для предсказания тех же 16 коэффициентов сплайна. Результат:  $R^2 < 0.6$ , недостаточный для качественного старта.

Причина: древовидные модели плохо аппроксимируют непрерывные нелинейные зависимости между физическими параметрами и функциональными весами. Только глубокие сети с гладкими активациями строят адекватную «карту решений».

### 5.2 Почему не PINNs

Physics-Informed Neural Networks решают уравнение напрямую, без численного солвера. Это отдельное направление с известными ограничениями:

- плохо работают при малых  $\varepsilon$  без специальных техник (см. PVD-ONet, 2026);
- медленная оптимизация для каждой новой задачи;
- нет гарантии точности решения.

Подход DeepBVP сохраняет все гарантии классического численного метода, добавляя лишь улучшенный старт.

## 6 Ограничения и направления развития

### 6.1 Текущие ограничения

- Модель обучена на конкретном классе уравнений (второй порядок, форма (1)). Другие типы BVP требуют отдельного обучения.
- При  $\varepsilon < 10^{-4}$  оба метода часто не сходятся — предел численного решателя, не модели.
- 16 В-сплайнов дают  $R^2 \approx 0.86$ ; для более резких погранслоёв может потребоваться адаптивный базис.

### 6.2 Направления развития

**Адаптивный базис.** Wavelet-преобразования или адаптивные узлы позволяют нейросети динамически сгущать базисные функции в области погранслоя.

**Физически-информированная дообучка (PINNs loss).** Добавление невязки уравнения в функцию потерь:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \cdot \|\varepsilon y'' + p y' + q y + j y^2 + k y^3 - f\|^2 \quad (7)$$

позволит улучшить качество без расширения датасета.

**Расширение класса уравнений.** При наличии нескольких специализированных моделей возможна интеграция с LLM в качестве классификатора типа уравнения — пользователь вводит произвольное уравнение, система маршрутизирует к нужной модели.

## 7 Воспроизводимость

Артефакты, необходимые для воспроизведения результатов, перечислены в таблице 6.

Таблица 6: Артефакты воспроизводимости

| Артефакт                            | Описание                                 |
|-------------------------------------|------------------------------------------|
| <code>generate_dataset_V4.py</code> | Генерация датасета, <code>seed=42</code> |
| <code>ML_V5_NN.py</code>            | Обучение модели, <code>seed=42</code>    |
| <code>test_V4.py</code>             | Тестирование, <code>seed=42</code>       |
| <code>app.py</code>                 | Streamlit веб-интерфейс                  |
| <code>model_nn_v5_turbo.pth</code>  | Веса модели                              |
| <code>scalers_v5.pkl</code>         | StandardScaler (X) + RobustScaler (Y)    |

**Окружение:** Python 3.10+, torch, scipy, numpy, sklearn, joblib, streamlit

**Демо:** <https://mlbvpsolver.streamlit.app/> - в стадии разработки